

ASSESSMENT TOOL

COURSE NAME: Theory of Computation

COURSE CODE: CSA13

COURSE OUTCOME COVERED

CO1: Analyze fundamental mathematical concepts of automata theory for formal languages and decision problems(BL:3)

Assessment Tool Weightage: 10%

QUESTIONS: 4

Total Marks:20

Duration : 1 Hour

Technical Presentation

Code of Conduct:

I C.MANO HAR/192372246 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Finite Automata Theory

NFA to DFA Conversion & Advanced Concepts

Presented by: C.Manohar | Reg No: 192372246

Theory of Computation (CSA1301) | Formal Languages & Automata

Problem 1: NFA to DFA Conversion

Converting NFA to DFA over alphabet $\Sigma=\{0,1\}$

Given NFA accepts language L over $\Sigma=\{0,1\}$. Construct equivalent DFA.

Original NFA States & Transitions:

- State A: $0 \rightarrow A$, $1 \rightarrow B$
- State B: $0 \rightarrow C$, $1 \rightarrow A$
- State C: $0 \rightarrow A$ (accepting), $1 \rightarrow B$
- Initial: A | Accepting: C

Equivalent DFA States:

- $q0 = \{A\}$
- $q1 = \{B\}$
- $q2 = \{C\}$
- $q3 = \{A, C\}$

DFA accepts strings ending at C

Challenges: Subset construction, state explosion, multiple paths

Problem 1: Subset Construction Method

1. Start with initial state: $\{A\}$
2. Create new DFA states as subsets of NFA states
3. For each new state & input symbol, compute transitions:
 - $\delta_{\text{DFA}}(\{A\}, 0) = \{A\}$ (where $0 \rightarrow A$ from A)
 - $\delta_{\text{DFA}}(\{A\}, 1) = \{B\}$ (where $1 \rightarrow B$ from A)
4. Repeat until no new states added
5. Mark states containing accepting NFA state C as accepting

Key Insight: DFA states are power sets of NFA states. May result in 2^n states.

THE UNIVERSITY OF CHICAGO
LIBRARY

1960

1960

1960

1960

1960

1960

Elimination of Epsilon (ϵ) Transitions

Role of ϵ -transitions in Finite Automata:

- Allow transitions without consuming input symbols
- Enable nondeterminism: multiple paths with same input
- Simplify automata construction & pattern matching
- Can be eliminated using epsilon-closure technique

ϵ -NFA to NFA Conversion:

1. Compute $\text{EPSILON-CLOSURE}(q)$ for each state q
 2. New transition $\delta'(q, a)$ includes all states reachable via:
 - Reading 'a' from q
 - Following ϵ -transitions from result
 3. State q is accepting if any state in $\text{EPSILON-CLOSURE}(q)$ is accepting
- 

Problem 3: Solution - Remove ϵ -Transitions

Given ϵ -NFA:

States: q_0, q_1, q_2

Initial: q_0

Accepting: q_2

Transitions:

- $q_0 \xrightarrow{\epsilon} q_1$
- $q_0 \xrightarrow{0} q_0$
- $q_1 \xrightarrow{\epsilon} q_2$
- $q_1 \xrightarrow{1} q_1$
- $q_2 \xrightarrow{0,1} q_2$

Transition Table:

State	$\delta(\cdot, 0)$	$\delta(\cdot, 1)$	Accepting
q_0	$\{q_0\}$	$\{q_1, q_2\}$	No
q_1	$\{q_2\}$	$\{q_1, q_2\}$	Yes
q_2	$\{q_2\}$	$\{q_2\}$	Yes

Epsilon Closures:

$$\text{EPSILON-CLOSURE}(q_0) = \{q_0, q_1, q_2\}$$

$$\text{EPSILON-CLOSURE}(q_1) = \{q_1, q_2\}$$

$$\text{EPSILON-CLOSURE}(q_2) = \{q_2\}$$

Equivalent NFA without ϵ :

$$\delta'(q_0, 0) = \{q_0\}$$

$$\delta'(q_0, 1) = \{q_1, q_2\}$$

$$\delta'(q_1, 0) = \{q_2\}$$

$$\delta'(q_1, 1) = \{q_1, q_2\}$$

$$\delta'(q_2, 0) = \{q_2\}$$

$$\delta'(q_2, 1) = \{q_2\}$$

Conversion

Computing ϵ -Closure for Each State:

Given NFA (from image):

States: A, B, C

Initial: A

Accepting: C

Transitions:

- $A \xrightarrow{\epsilon} B$
- $A \xrightarrow{0} A$
- $B \xrightarrow{\epsilon} C$
- $B \xrightarrow{1} B$
- $C \xrightarrow{0,1} C$

Equivalent DFA Construction:

Initial state: $\text{EPSILON-CLOSURE}(A) = \{A, B, C\} \rightarrow \text{Accepting!}$
From $\{A, B, C\}$ on 0: Go to $\text{EPSILON-CLOSURE}(\delta(\{A, B, C\}, 0)) = \text{EPSILON-CLOSURE}(\{A, C\}) = \{A, B, C\}$
From $\{A, B, C\}$ on 1: Go to $\text{EPSILON-CLOSURE}(\delta(\{A, B, C\}, 1)) = \text{EPSILON-CLOSURE}(\{B, C\}) = \{B, C\}$
From $\{B, C\}$ on 0: Go to $\text{EPSILON-CLOSURE}(\{C\}) = \{C\}$
From $\{B, C\}$ on 1: Go to $\text{EPSILON-CLOSURE}(\{B, C\}) = \{B, C\}$
From $\{C\}$ on 0,1: Goes to $\{C\}$

ϵ -Closure Calculation:

$\text{EPSILON-CLOSURE}(A):$

Start: $\{A\}$

$A \xrightarrow{\epsilon} B: \{A, B\}$

$B \xrightarrow{\epsilon} C: \{A, B, C\}$

$\text{EPSILON-CLOSURE}(B):$

$\{B, C\}$

$\text{EPSILON-CLOSURE}(C):$

$\{C\}$

Resulting DFA States:

$p0 = \{A, B, C\} \checkmark$

$p1 = \{B, C\}$

$p2 = \{C\}$

Initial: $p0$

Accepting: $p0, p1, p2$

Simplified Observations:

All reachable states are accepting! Language = $\{0, 1\}^*$ (all strings)

DFA Transition Table:

State	$\delta(q, 0)$	$\delta(q, 1)$	Accepting
$p0$	$p0$	$p1$	Yes $\checkmark$
$p1$	$p1$	$p1$	Yes $\checkmark$
$p2$	$p2$	$p2$	Yes $\checkmark$

Concepts & Takeaways

NFA vs DFA:

- ✓ NFA: Simpler to construct, multiple paths possible, accepts via any path
- ✓ DFA: Deterministic (single path), easier to implement, may have exponential states

Mathematical Induction in Automata:

- ✓ Proves correctness of automata properties and string acceptance
- ✓ Base case + inductive hypothesis $\rightarrow$ inductive step

Epsilon Transitions & Closure:

- ✓ ϵ -transitions enable free moves (no input consumed)
- ✓ Epsilon-closure determines all reachable states via ϵ -moves
- ✓ ϵ -NFA $\leftrightarrow$ NFA conversion uses subset construction with closures

All conversions preserve language acceptance: ϵ -NFA $\equiv$ NFA $\equiv$ DFA (same language L)

SIMATS ENGINEERING ASSESSMENT TOOL

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

92/100